

توسعه استریسک با برنامه نویسی AMI



VoiPing.ir

عناوین این بسته آموزشی

- معرفی کاربردهای توسعه استریسک با AMI
- بررسی دستورات کاربردی AMI در کنسول استریسک
- معرفی زبان‌ها و کتابخانه‌ها برای کار با AMI
- بررسی فایل‌های داخل پکیج آموزشی
- معرفی ابزارها و برنامه‌های استفاده شده
- معرفی راه‌های ارتباط با AMI
- AMI بر روی TCP
- اتصال به AMI از طریق Telnet
- AMI بر روی HTTP
- معرفی api و یا وب‌سرویس AJAM در مدل HTTP
- پی‌کربندی micro web server در استریسک
- اتصال به AMI از طریق Browser
- آشنایی با معماری AMI
- معماری کلی اینترفیس AMI
- ساختار Event ها
- ساختار Action ها
- ساختار Response ها
- پی‌کربندی AMI در استریسک
- بررسی سناریو و ساخت پایگاه داده
- ساخت ماژول تماس‌گیر خودکار (Manager Action)
- ساخت ماژول بررسی نتایج تماس‌های گرفته شده (Manager Event)

معرفی کاربردهای توسعه استریسک با AMI

- ساخت انواع نرم افزارهای موبایل و تحت وب برای مانیتورینگ مراکز تلفن
- ساخت انواع سیستم های تماس خودکار
- ساخت انواع سیستم های هشدار (مانند آتش سوزی و)
- اتصال سرور ویپ به CRM های مختلف
- ساخت انواع سیستم های یادآوری جلسات
- ساخت انواع سیستم های اعلام وضعیت
-

معرفی فایل‌های داخل پکیج آموزشی

- فایل‌های mp4 از محتویات دوره
- یک فایل Powr Point از محتویات دور
- یک فایل PDF از محتویات دوره
- پوشه voiping_senarios حاوی تمامی فایل‌های مربوط به سناریو
- کتابخانه phpagi.php و phpagi-asmanager.php
- کتابخانه AsterNet



معرفی برنامه‌های استفاده شده در این آموزش

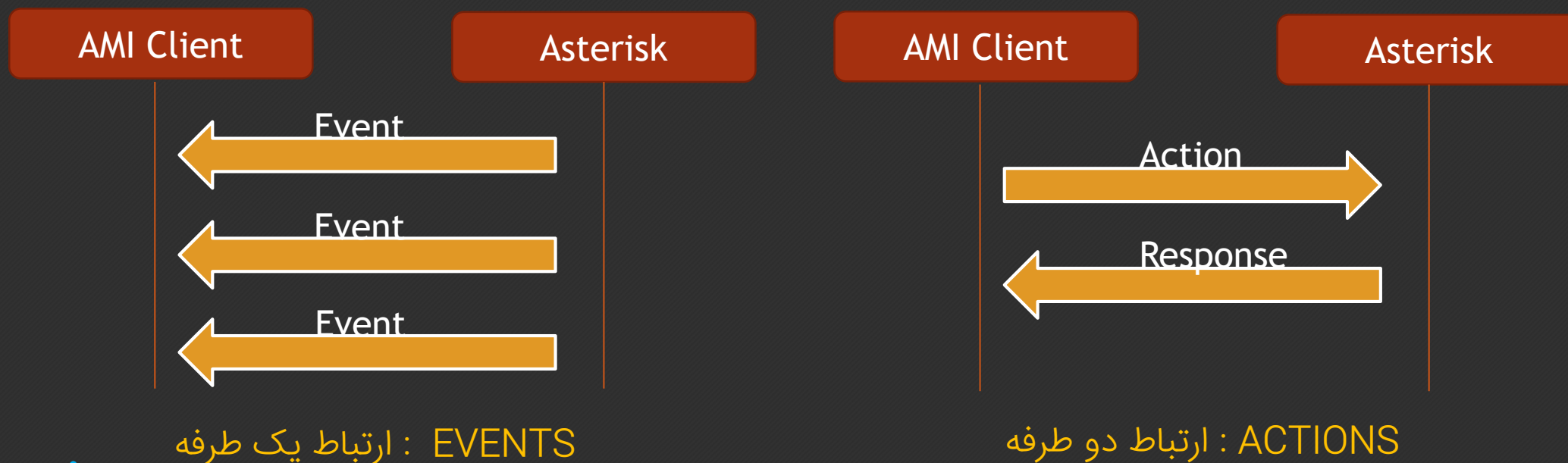
- برنامه WinSCP برای دسترسی آسان به فایل‌های سرور ویپ
- برنامه PuTTY برای اتصال به کنسول لینوکس و استریسک
- برنامه ++ Notepad برای ویرایش فایل‌ها و کدنویسی سناریو
- نرم افزار PHPMyAdmin برای کار با پایگاه داده

آشنایی با معماری AMI

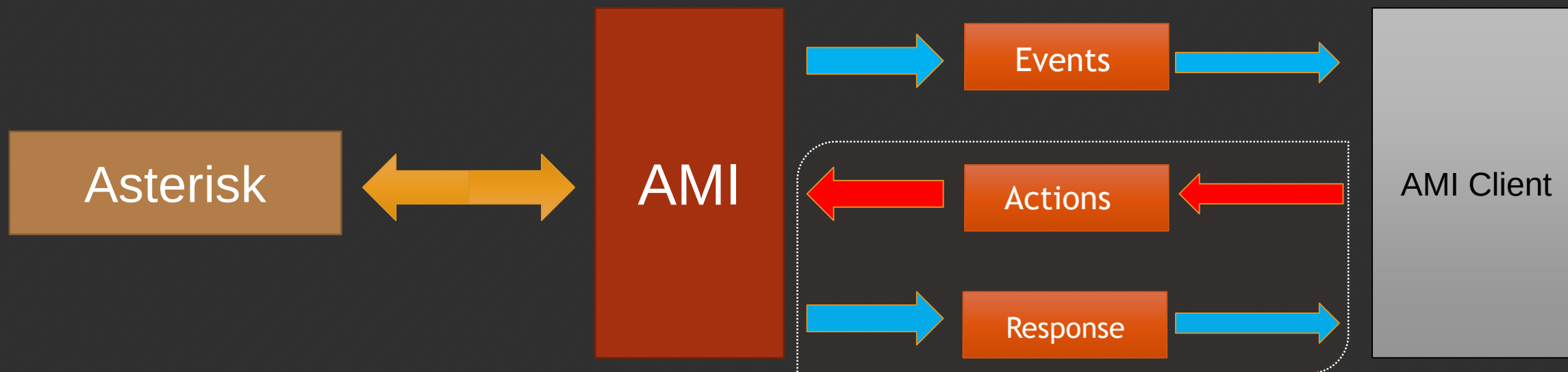
- AMI مخفف شده عبارت : Asterisk Manager Interface
- AMI ابزار مانیتورینگ استریسک می باشد .
- می توان به واسطه AMI از اتفاقات رخ داده در استریسک به صورت live مطلع شد .
- به وسیله AMI میتوان برخی از دستورات کاربردی را به استریسک برای اجرا ارسال نمود .
- تمامی فرآیندهای کاری در AMI به صورت Asynchronous در استریسک شکل می گیرد .
- به وسیله AMI می توان از هر شبکه ای با استریسک ارتباط برقرار کرد .
- پورت پیش فرض برای اتصال به AMI ، ۵۰۳۸ می باشد .
- حفظ امنیت و تنظیمات امنیتی در بحث AMI به شدت مورد نیاز می باشد .
- AMI یک سرویس مبتنی بر پیام می باشد .

آشنایی با معماری AMI

پیامها در AMI به دو نوع اصلی طبقه بندی می شوند: **ACTIONS** و **EVENTS**



آشنایی با معماری AMI



آشنایی با معماری AMI



زمانی که استریسک در پاسخ به یک ACTION یک لیست بر می‌گرداند ، نحوه تعامل با یوزر AMI به صورت زیر می‌باشد .

آشنایی با معماری AMI

ساختار پیام‌ها : `<key> : <value>`

هر خط از پیام‌های ارسالی با یک کاراکتر `\r\n` خاتمه می‌یابد و مجموعه هر دستور با دو کاراکتر `\r\n` خاتمه می‌یابد .

`<...> : <...> \r\n`

`<...> : <...> \r\n`

`<...> : <...> \r\n`

...

`\r\n\r\n`

آشنایی با معماری AMI



ساختار Event ها :

هر Event حداقل شامل یک هدر Event و یک هدر Privilege می باشد که اولی مشخص کننده نام رویداد و دومی مشخص کننده نوع کلاس این رویداد می باشد .

Event : Hangup
Privilege : call,all

....



آشنایی با معماری AMI

ساختار Action ها :

هر Action شامل یک هدر Action و دیگر آرگومان‌های متعلق به آن Action می‌باشد .
تقریباً هر تعاملی با AMI در استریسک باید با ACTION ای به نام LOGIN شروع شود .

Action : login
Username : <value>
Secret : <value>

آشنایی با معماری AMI

ساختار Response ها :

هر پاسخ حداقل شامل یک هدر Response که وضعیت اجرای Action در استریسک را مشخص می‌نماید و دیگری Message که پیام ارسالی از استریسک را نشان می‌دهد ، می‌باشد .

Action : login
Username : <value>
Secret : <value>



Response : Success
Message : Authentication accepted

Response :

- 1) Error
- 2) Success
- 3) Follows



آشنایی با معماری AMI

ساختار Respons ها :

زمانی که در پاسخ به یک Action یک لیست از اطلاعات باید برگردد ، ساختار آن به شکل زیر خواهد بود .

Action : CoreShowChannels



Response: Success
EventList: start
Message: Channels will follow
.....
Event: CoreShowChannelsComplete
EventList: Complete
ListItems: ...



پیکربندی AMI در استریسک

پیکر بندی AMI در دو سطح تنظیمات کلی و تنظیمات مربوط به کلاینت AMI در فایل مربوطه انجام می شود .

```
\etc\asterisk\manager.conf
```

```
[general]
```

```
....
```

```
[ami-username1]
```

```
....
```

```
[ami-username2]
```

```
....
```

پیکربندی AMI در استریسک

`\etc\asterisk\manager.conf`  `[general]`

Option	Value/Example	Description
enabled	yes	Enables the AMI. The default is no.
webenabled	yes	Allows access to the AMI through the built-in HTTP server. The default is no. ^[a]
port	5038	Sets the port number to listen on for AMI connections. The default is 5038.
bindaddr	127.0.0.1	Sets the address to listen on for AMI connections. The default is to listen on all addresses (0.0.0.0). However, it is highly recommended to set this to 127.0.0.1.
tlsenable	yes	Enables listening for AMI connections using TLS. The default is no. It is highly recommended to only expose connectivity via TLS outside of the local machine. ^[b]
tlsbindport	5039	Sets the port to listen on for TLS connections to the AMI. The default is 5039.
tlsbindaddr	0.0.0.0	Sets the address to listen on for TLS-based AMI connections. The default is to listen on all addresses (0.0.0.0).
tlscertfile	<code>/var/lib/asterisk/keys/asterisk.pem</code>	Sets the path to the server certificate for TLS. This is required if <code>tlsenable</code> is set to yes.
tlsprivatekey	<code>/var/lib/asterisk/keys/private.pem</code>	Sets the path to the private key for TLS. If this is not specified, the <code>tlscertfile</code> will be checked to see if it also contains the private key.

پیکربندی AMI در استریسک

`\etc\asterisk\manager.conf`  `[general]`

<code>tlscipher</code>	<code><cipher string></code>	Specifies a list of ciphers for OpenSSL to use. Setting this is optional. To see a list of available ciphers, run <i>openssl ciphers -v</i> at the command line.
<code>allowmultiplelogin</code>	<code>no</code>	Allows the same account to make more than one connection at the same time. The default is <code>yes</code> .
<code>displayconnects</code>	<code>yes</code>	Reports connections to the AMI as verbose messages printed to the Asterisk console. This is usually useful, but it can get in the way on a system that uses scripts that make a lot of connections to the AMI. The default is <code>yes</code> .
<code>timestampevents</code>	<code>no</code>	Adds a Unix epoch-based timestamp to every event reported to the AMI. The default is <code>no</code> .
<code>brokeneventsaction</code>	<code>no</code>	Restores previously broken behavior for the <code>Events</code> AMI action, where a response would not be sent in some circumstances. This option is there for the sake of backward-compatibility for applications that worked around a bug and should not be used unless absolutely necessary. The default is <code>no</code> .
<code>channelvars</code>	<code>VAR1,VAR2,VAR3[,VAR4[...]]</code>	Specifies a list of channel variables to include with all manager events that are channel-oriented. The default is to include no channel variables.
<code>debug</code>	<code>no</code>	Enables some additional debugging in the AMI code. This is primarily there for developers of the Asterisk C code. The default is <code>no</code> .

پیکربندی AMI در استریسک

`\etc\asterisk\manager.conf`  `[general]`

httptimeout	60	Sets the HTTP timeout, in seconds. This timeout affects users of the AMI over HTTP: it sets the <i>Max-Age</i> of the HTTP cookie, sets how long events are cached to allow retrieval of the events over HTTP using the <i>WaitEvents</i> action, and the amount of time that the HTTP server keeps a session alive after completing an AMI action. The default is 60 seconds.
-------------	----	--

[a] To access the AMI over HTTP, the built-in HTTP server must also be configured in `/etc/asterisk/http.conf`.

[b] The OpenSSL development package must be installed for Asterisk to be able to use encryption. On Ubuntu, the package is `libssl-dev`. On CentOS, the package is `openssl-devel`.

پیکربندی AMI در استریسک

`\etc\asterisk\manager.conf`  `[ami-username]`

Option	Value/Example	Description
secret	password	Sets the password used for authentication. This must be set.
deny	0.0.0.0/0.0.0.0	Sets an IP address Access Control List (ACL) for addresses that should be denied the ability to authenticate as this user. By default this option is not set.
permit	192.168.1.0/255.255.255.0	Sets an IP address ACL for addresses that should be allowed to authenticate as this user. As with deny, by default this option is not set. Without these options set, any IP address that can reach the AMI will be allowed to authenticate as this user.
writetimeout	100	Sets the timeout used by Asterisk when writing data to the AMI connection for this user. This option is specified in milliseconds. The default value is 100.
display connects	yes	Also available in the [general] section (refer to Table 20.1. "Options in the manager.conf [general] section"), but can be controlled on a per-user basis.
read	system,call[,...]	Defines which manager events this user will receive. By default, the user will receive no events. Table 20.3. "Available values for AMI user account read/write options" covers the available permission types for the read and write options.

پیکربندی AMI در استریسک

\etc\asterisk\manager.conf → [ami-username]

write	system,call[,...]	Defines which manager actions this user is allowed to execute. By default, the user will not be able to execute any actions. Table 20.3. “Available values for AMI user account read/write options” covers the available permission types for the read and write options.
eventfilter	!Channel: DAHDI*	Used to provide a whitelist- or blacklist-style filtering of manager events before they are delivered to the AMI client application. Filters are specified using a regular expression. A specified filter is a whitelist filter unless preceded by an exclamation point. [a]

[\[a\]](#) If no filters are specified, all events that are allowed based on the read option will be delivered. If only whitelist filters have been specified, only events that match one of the filters will be delivered. If there are only blacklist-style filters, all events that do not match any of the filters will be delivered. Finally, if there is a mix of whitelist- and blacklist-style filters, the whitelist filters will be processed first, and then the blacklist filters.

پیکربندی AMI در استریسک

`\etc\asterisk\manager.conf` → `[ami-username]` → read/write options

Permission identifier	read	write
all	Shorthand way of specifying that this user should have access to all available privilege options.	Grants user all privilege options.
system	Allows user to receive general system information, such as notifications of configuration reloads.	Allows user to perform system management commands such as <i>Restart</i> , <i>Reload</i> , or <i>Shutdown</i> .
call	Allows user to receive events about channels on the system.	Allows user to set information on channels.
log	Gives user access to logging information. [a]	read-only
verbose	Gives user access to verbose logging information. [b]	read-only
agent	Gives user access to events regarding the status of agents from the <code>app_queue</code> and <code>chan_agent</code> modules.	Enables user to perform actions for managing and retrieving the status of queues and agents.
user	Grants access to user-defined events, as well as events about Jabber/XMPP users.	Lets user perform the <code>UserEvent</code> manager action, which provides the ability to request that Asterisk generate a user-defined event. [c]

پیکربندی AMI در استریسک

`\etc\asterisk\manager.conf` → `[ami-username]` → read/write options

config	write-only	Allows user to retrieve, update, and reload configuration files.
command	write-only	Allows user to execute Asterisk CLI commands over the AMI.
dtmf	Allows user to receive events generated as DTMF passes through the Asterisk core. [d]	read-only
reporting	Gives user access to call-quality events, such as jitterbuffer statistics or RTCP reports.	Enables user to execute a range of actions to retrieve statistics and status information from across the system.
cdr	Grants user access to CDR records reported by the <code>cdr_manager</code> module.	read-only
dialplan	Allows user to receive events generated when variables are set or new extensions are created.	read-only
originate	write-only	Allows user to execute the <code>Originate</code> action, which allows an AMI client to request that Asterisk create a new call.

پیکربندی AMI در استریسک

`\etc\asterisk\manager.conf` → `[ami-username]` → read/write options

cc	Allows user to receive events related to Call Completion Supplementary Services (CCSS).	read-only
aoc	Lets user see Advice of Charge events generated as AOC events are received.	Allows user to execute the <code>AOCMessage</code> manager action, for sending out AOC messages.

[a] This level has been defined, but it is not currently used anywhere in Asterisk.

[b] This level has been defined, but it is not currently used anywhere in Asterisk.

[c] The `UserEvent` action is a useful mechanism for having messages delivered to other AMI clients.

[d] DTMF events will not be generated in a bridged call between two channels unless generic bridging in the Asterisk core is being used. For example, if the DTMF is being transmitted with the media stream and the media stream is flowing directly between the two endpoints, Asterisk will not be able to report the DTMF events.



پیکربندی AMI در استریسک

برای اطمینان از لود شدن کامل ماژول های استریسک ، به هنگام اتصال به اینترفیس AMI ، استریسک Event زیر را برای اطمینان ، ارسال می نماید .

Event: FullyBooted
Privilege: system,all
Status: Fully Booted

دستورات کاربردی برای AMI در محیط کنسول استریسک

- CLI >>manager show settings
- CLI >>manager show users
- CLI >>manager show user <user>
- CLI >>manager show connected
- CLI >>manager show events
- CLI >>manager show event <event>
- CLI >>manager show commands
- CLI >>manager show command <actions>



معرفی راه‌های ارتباط با AMI

برای اتصال به اینترفیس AMI می‌توان از دو روش زیر استفاده نمود .

۱. AMI over TCP/IP
۲. AMI over HTTP



معرفی راه‌های ارتباط با AMI

اتصال از طریق telnet در مدل TCP/IP :

از طریق یک نرم افزار به سرویس استریسک با پورت ۵۰۳۸ ارتباط برقرار می‌نماییم و اولین action خود ، login را اجرا می‌کنیم :

```
action: login  
username: voiping  
secret: voiping  
event: on
```



معرفی راه‌های ارتباط با AMI

اتصال در مدل HTTP :

Asynchronous Javascript Asterisk Manager (AJAM)

- AJAM تکنولوژی معرفی شده توسط استریسک از ورژن ۱.۴ می‌باشد که هدف آن فراهم کردن شرایط ارتباطی بین اپلیکیشن‌های مبتنی بر HTTP با اینترفیس AMI در استریسک بود .
- این راه‌کار بر پایه تکنولوژی Ajax می‌باشد .

معرفی راه‌های ارتباط با AMI

اتصال در مدل HTTP :

ابتدا مینی وب سرور مخصوص استریسک را با تنظیمات مربوطه در فایل http.conf فعال می‌نماییم و سپس به این وب سرور اجازه دسترسی به AMI را با تنظیمات مربوطه در فایل manager.conf می‌دهیم :

/etc/asterisk/http.conf

[general]

enabled=yes

bindaddr=0.0.0.0

bindport=8088

prifix=

enablestatic=

/etc/asterisk/manager.conf

[general]

webenabled = yes



معرفی راه‌های ارتباط با AMI

اتصال از طریق Browser در مدل HTTP :

بررسی وضعیت پیکربندی سرویس HTTP در استریسک :

<http://ip:8088/httpstatus>

: برخی از دموهای ami

<http://ip:8088/manager>

<http://ip:8088/amanager>

<http://ip:8088/mxml?action=...>

<http://ip:8088/amxml?action=...>

<http://ip:8088/rawman?action=...>

<http://ip:8088/arawman?action=...>

<http://ip:8088/static/ajamdemo.html>

<http://ip:8088/static/mantest.html>



معرفی راه‌های ارتباط با AMI

اتصال از طریق Browser در مدل HTTP :

در مدل HTTP برای واکنشی رویدادها از AMI حتما باید از ACTION ای به نام WaitEvent استفاده کرد . Response این ACTION همان EVENT های مد نظر ما خواهد بود . این روش تنها راهکار برای خواندن رویدادها در مدل HTTP می‌باشد .

نکته بسیار با اهمیت :

استفاده از کتابخانه‌های زبان‌های برنامه نویسی تحت وب ، مانند PHP ، برای کار با AMI ، به معنی استفاده از مدل HTTP نمی‌باشد .

این کتابخانه‌ها برای اتصال به AMI در برنامه‌های تحت وب از تکنولوژی Socket استفاده می‌کنند ، که این روش همان مدل TCP/IP خواهد بود . بنابراین نکته‌ای که در بالاتر به آن اشاره شد ، در این سناریوها فاقد اعتبار خواهد بود .

معرفی زبان‌ها و کتابخانه‌ها برای کار با AMI

زبان‌های رایج برنامه نویسی که می‌توان در AMI از آن‌ها استفاده کرد :

Php , Python , Ruby, Node js , .Net , JAVA

Name	Language	Website
Adhearsion	Ruby	http://www.adhearsion.com/
Asterisk-Java	Java	https://blogs.reucon.com/asterisk-java/
PHPAGI	PHP	http://phpagi.sourceforge.net/
Pyst۲	Python	https://github.com/rdegges/pyst۲
AsterNET	.NET (C#/VB.net)	https://github.com/skrusty/AsterNET
IO	node.js	https://github.com/zmarius۸۱/asterisk.io

معرفی کتابخانه PHPManager برای کار با AMI در PHP

زمانی که ایزوهای مبتنی بر FreePBX را نصب می‌نمایید این کتابخانه در مسیر پیشفرض AGI ها قابل دسترس می‌باشد :

`/var/lib/asterisk/agi-bin/phpagi-asmanager.php`

این کتابخانه بر پایه مدل شی‌گرا توسعه داده شده است .

این کتابخانه برای ارتباط با AMI از تکنولوژی Socket در مدل TCP/IP استفاده می‌نماید .



معرفی کتابخانه AsterNet برای کار با AMI در .net

این کتابخانه بر پایه مدل شی‌گرا توسعه داده شده است .
این کتابخانه برای ارتباط با AMI از تکنولوژی Socket در مدل TCP/IP استفاده می‌نماید .
این کتابخانه ، هم برای پشتیبانی از FastAGI و هم Async AGI و هم برای اتصال به AMI در تمامی زبان‌های .net توسعه داده شده است .

<https://github.com/AsterNET/AsterNET>

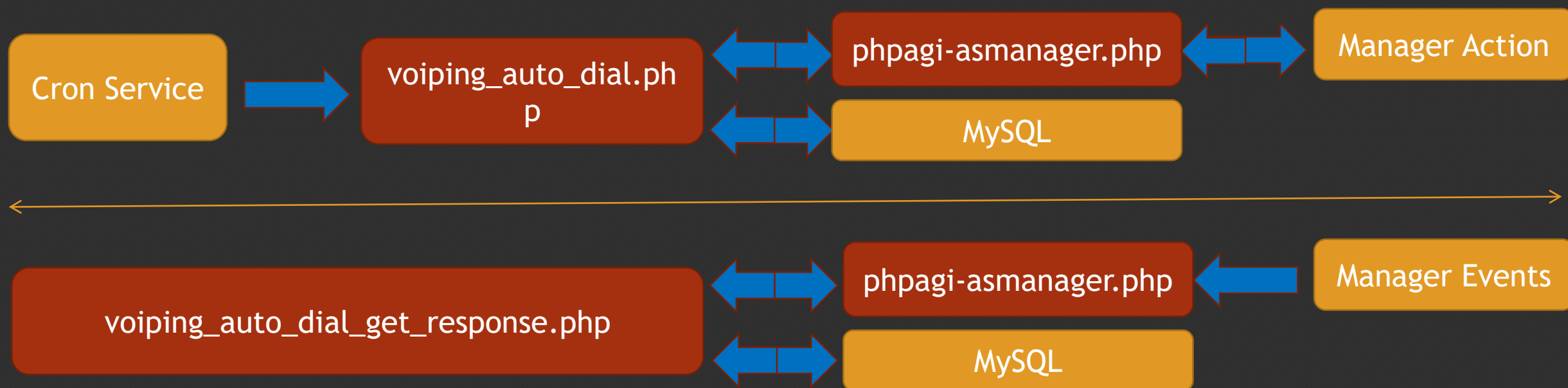
<https://www.nuget.org/packages/AsterNET/>

<http://asternet.github.io/AsterNET/html/79b6241e-05a3-441c-b6a1-51f2b5b7f265.htm>

ساخت ماژول تماس خودکار

- این ماژول در **بازه‌های زمانی مشخصی** (با قابلیت تنظیم) به پایگاه داده ما مراجعه کرده و **تعداد مشخصی** (با قابلیت تنظیم) شماره تلفن را واکشی کرده .
- این ماژول با تک تک شماره‌ها تماس گرفته و پس از دریافت پاسخ ، مخاطب را به یک IVR هدایت خواهد کرد .
- **موفقیت و یا عدم موفقیت** برای هر تماس ، در **پایگاه داده ذخیره** خواهد شد .
- برای مدیریت بازه‌های زمانی مشخص ، از سیستم CRON در لینوکس استفاده خواهد شد .
- در پیاده سازی این ماژول از کتابخانه PHPManager استفاده خواهد شد .

ساخت ماژول تماس خودکار



ساخت ماژول تماس خودکار



Originate :

Action: Originate
ActionID: <value>
Channel: <value>
Exten: <value>
Context: <value>
Priority: <value>
Application: <value>
Data: <value>

Timeout: <value>
CallerID: <value>
Variable: <value>
Account: <value>
EarlyMedia: <value>
Async: <value>
Codecs: <value>

```
$asm->send_request('Originate',  
    array('Channel'=>"$trunk/$number",  
        'Context'=>"$context",  
        'priority'=>"$priority",  
        'exten'=>"$exten",  
        'Async'=>'yes',  
        'ActionID'=>'voiping_auto_dial_'."$id",  
        'callerid'=>"$callerid"));
```

ساخت ماژول تماس خودکار



OriginateResponse :

[ActionID:] <value>

Response: <value>

Channel: <value>

Context: <value>

Exten: <value>

Application: <value>

Data: <value>

Reason: <value>

Uniqueid: <value>

CallerIDNum: <value>

CallerIDName: <value>

```
$asm->add_event_handler("OriginateResponse", "callback");  
function callback($event,$data){  
    ...  
}  
$asm->wait_response();
```



ساخت ماژول تماس خودکار

- چرا باید از سرویس CRON در سیستم‌های تماس خودکار استفاده کنیم ؟
- آیا با وابستگی به یک فایل دوم که باید همیشه در حال اجرا باشد ، می‌توان یک ماژول برای فروش به بازار داشته باشیم ؟
- آیا این دو فایل PHP را می‌توانستیم به یک فایل تبدیل نماییم ؟
- آیا در این حد از کد نویسی ، ماژول ما آماده ارائه به بازار برای فروش می‌باشد ؟

با تمرین زیاد به آسانی یاد بگیریم



VoiPing.ir